

Kubernetes Authorization

Since you already understand **Authentication**, let's connect it with Authorization.

1. What is Authorization?

Authorization is the process of checking what an authenticated user or service is allowed to do in Kubernetes.

In simple words:

Authentication = Who are you? 

Authorization = What are you allowed to do? 

Example

Suppose you are authenticated as:

User = Abeera

Kubernetes then checks:

Can Abeera:

 View Pods?

 Create Pods?

 Delete Nodes?

 Change cluster settings?

That permission checking is **Authorization**.

2. Why Do We Need Authorization?

Imagine Kubernetes had authentication but **no authorization**.

Anyone who successfully connects could potentially perform actions they should not be allowed to perform.

For example:

Developer



Authenticated 



Can delete Nodes? 

Can view Pods? 

Authorization prevents users from getting **more permissions than they need**.

Main purpose

 **Control access to Kubernetes resources.**

3. Where Does Authorization Happen?

Remember the Kubernetes API Server:

User



API Server



|— Authentication



| Who are you?



|— Authorization



What can you do?

The **API Server** checks authorization for API requests.

4. Authentication vs Authorization

Authentication

Verifies identity

"Who are you?"

Happens first

Example: certificate/token

Authorization

Verifies permissions

"What can you do?"

Happens after authentication

Example: RBAC rule

Easy trick:



AUTHENTICATION = WHO



AUTHORIZATION = WHAT

5. Types of Kubernetes Authorization

Kubernetes supports several authorization modes.

The important ones to know are:

1. **Node**
2. **ABAC**
3. **RBAC** ★
4. **Webhook**

You may also see:

5. **AlwaysAllow**
6. **AlwaysDeny**

Let's understand each one.

6. Node Authorization

What is it?

Node authorization is specifically designed to authorize requests made by **Kubernetes Nodes/Kubelets**.

Remember:

Worker Node

|



Kubelet

|



API Server

The Kubelet needs access to certain Kubernetes resources.

Node authorization controls what a Node is allowed to access.

Purpose

 **Control what Nodes/Kubelets can access.**

Example

A Kubelet on Node 1 should be able to access information needed to manage workloads on Node 1.

It should **not automatically have unlimited access to everything in the cluster.**

Use case

Useful for:

- Kubelet-to-API-server authorization
- Restricting Node access
- Protecting cluster resources from overly broad Node permissions

Easy memory:

 **Node Authorization = Permissions for Nodes/Kubelets**

7. ABAC

ABAC means:

Attribute-Based Access Control

It uses **attributes and policies** to determine whether a request is allowed.

Think of a rule such as:

User = Developer

Resource = Pod

Action = get

A policy can determine whether this combination is allowed.

Purpose

 Control access using policy attributes.

Example concept

Developer

+

Pod

+

Read

↓

Allowed 

But:

Developer

+

Node

+

Delete

↓

Denied 

Use case

ABAC can be used when authorization decisions are based on policy attributes.

Important KCNA point ★

ABAC exists in Kubernetes, but RBAC is much more commonly used in modern Kubernetes environments.

Easy memory:

🏷️ **ABAC = Access based on Attributes**

8. RBAC ★★ ★

RBAC means:

Role-Based Access Control

This is one of the **most important authorization concepts in Kubernetes.**

Instead of giving permissions directly to every user, we create **Roles** containing permissions and assign those roles to users or other identities.

Easy example

Developer



Role



Can:

✅ get Pods

✅ list Pods

❌ delete Nodes

9. Why Use RBAC?

Imagine you have 100 developers.

You don't want to manually create permissions for every developer.

Instead:

Role = Developer

Give the role to developers.

Developer 1 —┐

Developer 2 —┴—→ Developer Role

Developer 3 —┐

Now they receive the permissions defined by that role.

Purpose



Give users only the permissions they need.

This follows the principle of:

Least privilege

Meaning:

Give a user the minimum permissions necessary to do their job.

10. RBAC Example

Suppose a developer needs to view Pods.

We can create a Role that allows:

Resource: Pods

Verb: get

Verb: list

Then connect that Role to the developer.

Developer

|



Role

|

└─ get Pods

└─ list Pods

The developer can view Pods but doesn't automatically get permission to delete Nodes.

11. RBAC Components

For KCNA, remember these four terms:

Role

RoleBinding

ClusterRole

ClusterRoleBinding

Role

Defines permissions **inside a namespace**.

Example:

Role

↓

Can get Pods

Can list Pods

RoleBinding

Connects a **Role to a user/group/service account**.

User

↓

RoleBinding

↓

Role

ClusterRole

Defines permissions that can apply at the **cluster level** or can be used for namespaced resources depending on the binding.

ClusterRoleBinding

Connects a ClusterRole to a user/group/service account **across the cluster**.

User



ClusterRoleBinding



ClusterRole

12. Webhook Authorization

What is a Webhook?

A Webhook allows Kubernetes to ask an **external authorization service**:

"Should I allow this request?"

Flow:

User



API Server



External Authorization Service



└─ Deny ❌

Purpose

🌐 Use an external system to make authorization decisions.

Use case

An organization may already have its own authorization system and want Kubernetes to consult that system.

Easy memory:

🌐 Webhook = Ask an external service

13. AlwaysAllow

This authorization mode basically says:

Allow all authorization requests.

Request



AlwaysAllow



✅ ALLOWED

Purpose

Very simple authorization behavior.

Important

It is **not appropriate for a secure production environment** because it does not restrict access.

Memory:

🟢 AlwaysAllow = Everyone allowed

14. AlwaysDeny

This is the opposite.

Deny all authorization requests.

Request



AlwaysDeny



 DENIED

Memory:

 **AlwaysDeny = Everyone denied**

It can be useful for testing or specific configurations, but it would prevent normal access if used by itself.

15. Comparison of Authorization Types

Type	Easy Meaning	Main Use
Node	Controls Node/Kubelet access	Node authorization
ABAC	Uses attributes/policies	Attribute-based access
RBAC 	Uses Roles	Most common general authorization approach
Webhook	Asks external service	External authorization
AlwaysAllow	Allows everything	Simple/testing scenarios
AlwaysDeny	Denies everything	Testing/restrictive scenarios

16. Easy Real-Life Example

Imagine an office.

Authentication

Security guard asks:

"Who are you?"

You show your ID.

Authentication 

Now the office asks:

"What rooms can you enter?"

That's:

Authorization 

RBAC

You are a:

Developer

Your **Developer Role** allows:

Office 

Computer Room 

Server Room 

Webhook

The office calls an external security system:

"Is this person allowed?"

External system responds:

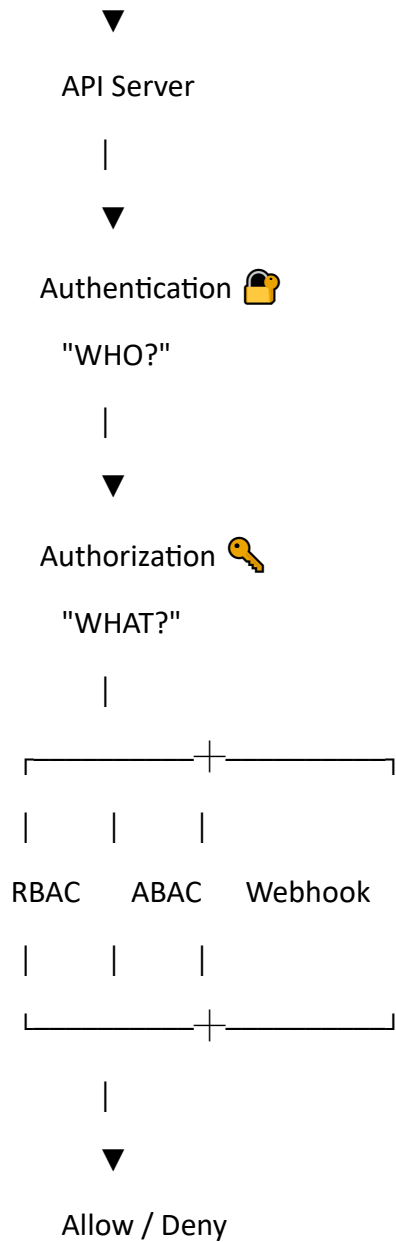
YES 

17. Complete Kubernetes Security Flow

This is the flow I want you to remember:

User

|



18. Most Important for KCNA ★

Don't try to memorize everything equally.

Focus strongly on:

1 Authentication

WHO are you?

2 Authorization

WHAT can you do?

3 RBAC

WHO can do WHAT?

4 Role

What permissions are defined?

5 RoleBinding

Who gets those permissions?

6 Webhook

Ask an external authorization service.

Super Easy Revision Notes

AUTHENTICATION



WHO are you? 

AUTHORIZATION



WHAT can you do? 

Authorization Types:

Node



Kubelet/Node permissions

ABAC



Attribute-based policies

RBAC ★



Role-based permissions

Webhook



External authorization service

AlwaysAllow



Allow everything

AlwaysDeny



Deny everything

★ **One-line definition for exam:**

Kubernetes Authorization determines whether an authenticated identity is allowed to perform a requested action on a Kubernetes resource.